Assignment 5 – Sudoku Solver

Updated UML Use Case Diagram and Use Case Documentation | EECS 348 – Software Engineering I

Updated for Assignment 5

This document updates the inherited Assignment 4 use case model to match the Assignment 5 program. The primary actors and overall workflow are retained. The key functional change is that completed Sudoku solutions are written to separate output files named solution1.txt through solution5.txt instead of being displayed as solution grids in the console.

1. Updated UML Use Case Diagram

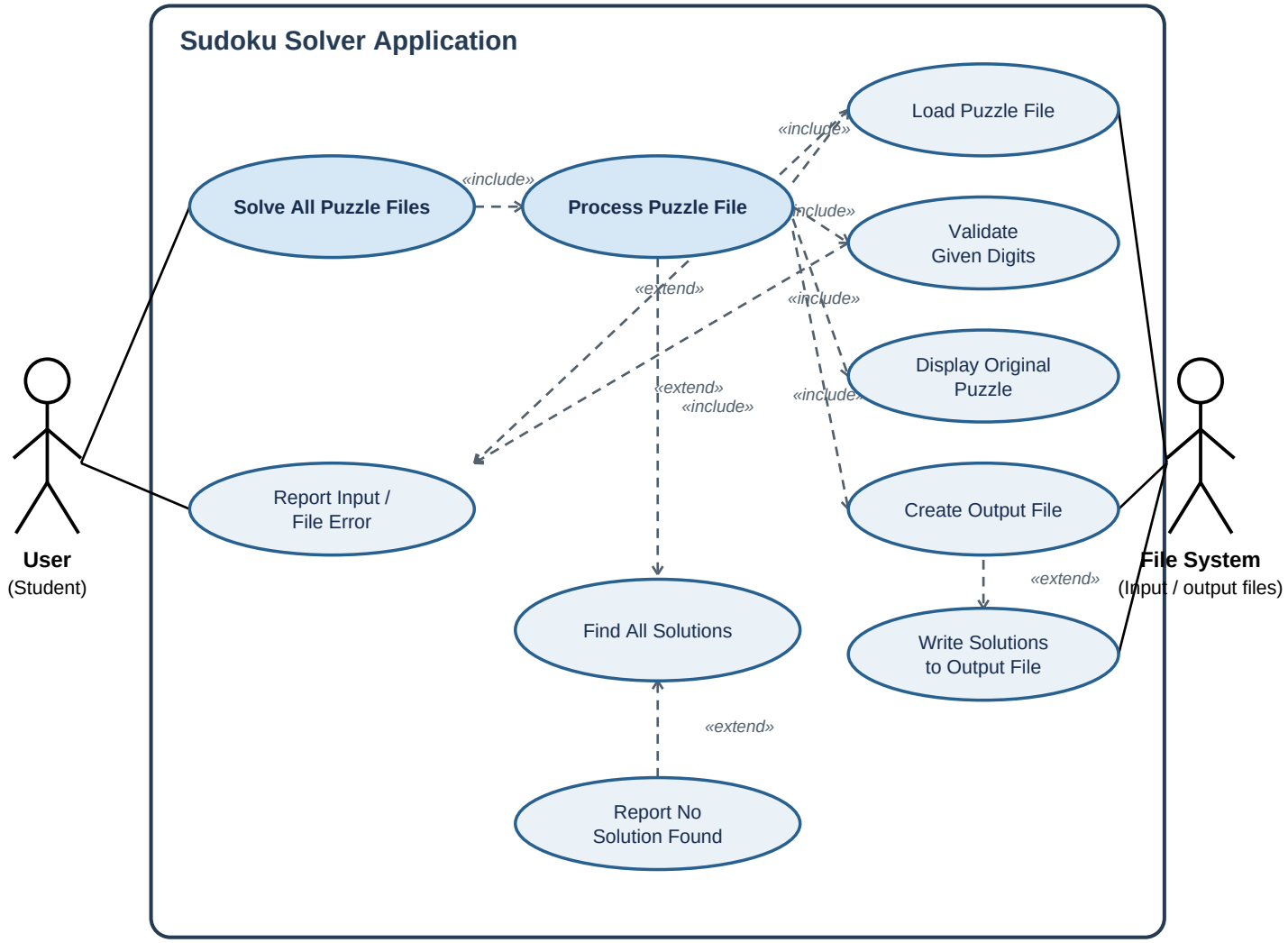

Diagram notes: The User starts the application. The File System supplies the five input files and receives the five solution files. Process Puzzle File is repeated for each input. Write Solutions to Output File occurs when one or more solutions are found; Report No Solution Found occurs when the search finds none. Input and file errors are reported when loading, validation, or output-file operations fail.

2. Use Case Documentation

The following use cases retain the inherited Assignment 4 model while updating the output behavior for Assignment 5. The main use case is UC-1. The remaining use cases describe the supporting behavior shown in the diagram. The actor names and system scope are based on the supplied Assignment 4 document.

UC-1: Solve All Puzzle Files

Actors: Primary actor: User (student). Secondary actor: File System.

Brief description

The User runs the Sudoku Solver Application. The application processes puzzle1.txt through puzzle5.txt, displays each original puzzle, searches for every valid completion, and writes the results to the matching solutionN.txt file.

Step-by-step flow

1. The User runs Assignment5.exe from the directory containing the puzzle files.
2. The application selects the next input file, beginning with puzzle1.txt.
3. The application loads the puzzle and checks the input format.
4. The application displays the original puzzle in the console.
5. The application validates the given digits.
6. The application creates the matching solutionN.txt output file.
7. The application searches for every valid solution using depth-first search and backtracking.
8. The application writes all completed grids and the solution count to the output file, or writes the no-solution message.
9. The application repeats the process for the remaining puzzle files and then exits.

Detailed description

The User triggers this use case by launching the application. The File System provides the five named puzzle files. For each file, the application reads exactly 81 non-whitespace cell characters, where digits 1 through 9 represent given values and an underscore represents an empty cell. If loading succeeds, the original grid is displayed and the given digits are checked for conflicts. The application then opens the corresponding solution output file and runs the recursive search. Every completed grid is written to that file, followed by a line stating 'Solutions found: n'. If the puzzle has no valid completion, the output file receives 'No solution found.' A missing or malformed input file is reported and skipped. An output-file creation or writing failure is reported. The application continues with later puzzle files where its control flow permits and returns a nonzero status if a file-processing error occurred.

UC-2: Process Puzzle File

Actors: Primary actor: User. Supporting actor: File System.

Brief description

Processes one puzzle file from input loading through solution output. UC-1 repeats this use case for each of the five files.

Step-by-step flow

1. UC-1 selects the next puzzle filename.
2. The application loads the selected file.
3. The application displays the original puzzle after successful loading.
4. The application validates the given digits.
5. The application creates the matching solution output file.
6. The application searches for all solutions and writes the result.
7. The application returns to UC-1 to process the next file.

Detailed description

This use case represents one complete iteration of the application's five-file loop. It coordinates the smaller use cases for loading, validating, displaying, creating the output file, and solving the puzzle. If the input file cannot be loaded, the application reports the input error and skips that file. If the output file cannot be created or written, the application reports the file error and records an overall error status. Processing then continues with the next input file when possible.

UC-3: Load Puzzle File

Actors: Primary actor: File System. Initiating actor: User through UC-1.

Brief description

Reads one puzzle file and converts its 81 cell characters into the application's 9x9 Sudoku grid.

Step-by-step flow

1. The application opens the selected puzzle file.
2. The application reads non-whitespace characters one at a time.
3. Each underscore is stored as an empty cell and each digit from 1 through 9 is stored as a given value.
4. The application checks for invalid characters and extra data.
5. The application verifies that exactly 81 cells were read.
6. The application stores the parsed grid when the input is valid; otherwise, it returns an error.

Detailed description

The application requests the named input file from the File System. It reads each non-whitespace character and maps underscores to empty cells and digits 1 through 9 to integer values in the grid. It rejects invalid characters, more than 81 cells, fewer than 81 cells, and read failures. The parsed grid is copied into the puzzle only after the entire input has been read successfully, preventing a partially read file from becoming the active puzzle.

UC-4: Validate Given Digits

Actors: Primary actor: Sudoku Solver Application. Supporting actor: User indirectly through UC-1.

Brief description

Checks that the originally supplied digits do not conflict in any row, column, or 3x3 box.

Step-by-step flow

1. The application examines each cell in the loaded grid.
2. For each nonempty cell, it checks the other cells in the same row.
3. It checks the other cells in the same column.
4. It checks the other cells in the same 3x3 box.
5. If a duplicate is found, validation fails and an input/conflict message is reported.
6. If no conflicts are found, processing continues to the solving stage.

Detailed description

Validation examines every nonempty cell in the original puzzle. For each given value, the application checks whether the same value appears elsewhere in that cell's row, column, or 3x3 box. A repeated value means the puzzle contains conflicting givens, so the application does not begin the search for that puzzle. In the current Assignment 5 implementation, the solve operation writes 'No solution found.' to the already opened solution output stream when validation fails. The application also treats the conflict as an invalid puzzle condition.

UC-5: Display Original Puzzle

Actors: Primary actor: User.

Brief description

Displays the loaded puzzle in the console before the solver searches for solutions.

Step-by-step flow

1. The application successfully loads a puzzle file.
2. The application prints the label 'Original puzzle:'.
3. The application prints the 9x9 grid.
4. Empty cells are displayed as underscores.
5. Processing continues to output-file creation and solving.

Detailed description

After a puzzle file is loaded successfully, the application displays the original grid in the console. Each row contains nine cell values separated by spaces. Empty cells are represented by underscores. This use case displays the input puzzle for the User; it does not display the completed solution grids in the console. Completed solutions are written to the corresponding solutionN.txt file.

UC-6: Find All Solutions

Actors: Primary actor: Sudoku Solver Application.

Brief description

Uses recursive depth-first search with backtracking to find every valid completion of the loaded Sudoku puzzle.

Step-by-step flow

1. The application starts with the loaded puzzle grid.
2. The search identifies empty cells and calculates legal candidates for each.
3. If an empty cell has no legal candidates, the current search branch ends.
4. The search selects the empty cell with the fewest legal candidates.
5. The search places each candidate in turn and recursively explores the resulting grid.
6. When a complete grid is found, the solution is recorded through the output stream.
7. The search backtracks and continues until every candidate branch has been explored.

Detailed description

The solver uses recursive depth-first search and backtracking. At each recursive call, it scans the empty cells and determines which digits can legally be placed based on the row, column, and 3x3 box rules. It uses the Minimum Remaining Values approach by selecting the empty cell with the fewest legal candidates. If any empty cell has no candidate, that branch is abandoned. If no empty cells remain, a complete solution has been found and the solution counter is increased. The solver then returns to the previous call, clears the trial value, and continues exploring other candidates. This continues until all possible branches have been considered.

UC-7: Write Solutions to Output File

Actors: Primary actor: File System. Supporting actor: Sudoku Solver Application.

Brief description

Writes every completed Sudoku grid to the output file associated with the input puzzle, followed by the total solution count.

Step-by-step flow

1. The application creates the corresponding solutionN.txt file.
2. The search finds a completed valid Sudoku grid.
3. The application writes the 9x9 grid using spaces between cells and underscores for empty cells (none in a completed grid).
4. The application writes a blank line between completed grids.
5. The search continues until all possible solutions have been found.
6. The application writes 'Solutions found: n' as the final result line.
7. The application closes the output file and checks for stream errors.

Detailed description

This use case replaces the inherited Assignment 4 behavior of displaying completed solutions in the console. The Assignment 5 application opens a separate output file for each input puzzle: puzzle1.txt corresponds to solution1.txt, and the same numbering applies through puzzle5.txt and solution5.txt. Whenever the recursive search finds a complete grid, it writes that grid to the output stream. After all search branches have been explored, the application writes the total number of solutions. The output stream is closed after solving. If a write or close operation fails, the application catches the stream failure and reports an output-file error.

UC-8: Report No Solution Found

Actors: Primary actor: User through the output file; supporting actor: File System.

Brief description

Records that a puzzle has no valid completed grid.

Step-by-step flow

1. The application finishes the search without finding a completed grid, or validation rejects conflicting givens.
2. The application writes 'No solution found.' to the corresponding solution output file.
3. The application finishes processing that puzzle and continues with the next file.

Detailed description

This conditional use case occurs when the solver cannot find any valid completion. It also occurs in the current implementation when the original givens conflict, because solveAll writes the same message after validation fails. The exact message is written as a single line to the corresponding solutionN.txt file. A puzzle with no solution is a valid search result and is distinct from a missing or malformed input file.

UC-9: Report Input / File Error

Actors: Primary actor: User. Supporting actor: File System.

Brief description

Reports a problem that prevents a puzzle from being loaded or its result from being saved.

Step-by-step flow

1. The application attempts to open or read an input puzzle file, or create/write its output file.
2. An input or output operation fails, or the input format is invalid.
3. The application creates an appropriate error message.
4. The application displays the error message in the console.
5. The application records an error status and continues with the next puzzle when possible.
6. The application returns a nonzero status if any file-processing error occurred.

Detailed description

This use case handles missing input files, invalid characters, incorrect cell counts, input read failures, output file creation failures, and output writing or closing failures. Input loading errors are reported to the console and the affected puzzle is skipped. Output-file errors are also reported to the console and set the application's error flag. The program continues processing later puzzles where possible. At the end of the run, the application returns status 1 if an input or output file-processing error occurred; otherwise it returns status 0. A puzzle that simply has no solution does not by itself cause an error exit status.

3. Relationships and Implementation Notes

Relationship	Description
UC-1 includes UC-2	The main use case processes each of the five puzzle files.
UC-2 includes UC-3, UC-4, UC-5, UC-6	Each puzzle is loaded, validated, displayed, and searched.
UC-2 includes output-file creation	The corresponding solutionN.txt file is opened before solving.
UC-6 extends to UC-7	When solutions are found, each completed grid is written to the output file.
UC-6 extends to UC-8	When no completion is found, the no-solution message is written.
UC-3 / UC-4 / output operations extend to UC-9	Errors are reported when input loading, validation, or output-file operations fail.

Assumptions and scope

- The actors and original use case names are based on the uploaded Assignment 4 diagram and its four pages of documentation.
- The updated output behavior is based on the Assignment 5 C++ code supplied in this conversation: solution grids and solution counts are written to solution1.txt through solution5.txt, not printed to the console.
- The original puzzle is still displayed in the console after successful input loading.
- The File System is shown as a secondary actor because it supplies input files and receives output files.
- The diagram keeps the inherited system boundary and primary User actor. The output-file use case is renamed to reflect Assignment 5.
- The current code reports conflicting givens through the no-solution message in the output file. The inherited Assignment 4 description treated this as an input error; the documentation above distinguishes the conflict condition while reflecting the current code's output behavior.
- No new Sudoku features, such as editing puzzles, saving input changes, or interactive cell entry, are included because they are not supported by the supplied program.

Prepared for EECS 348 Assignment 5 | October 29, 2026